調査レポート: poc-investigation-targets

生成日時: 2026-06-02

エグゼクティブサマリー

調査対象

項目	内容
プロジェクト名	poc-investigation-targets
サブシステム①	legacy-order-inventory-system（Java/Servlet/JSP + Oracle 10g の老
サブシステム②	asteria-etl-sample（ASTERIA Warp 1709 ベースの基幹↔DWH ETL）
調査日	2026-06-02
GitHub リポジトリ	baoshi1970/poc-investigation-targets

発見した問題の総数と重大度別集計

重大度	件数	主な内容
Critical（即時対応）	4 件	DB接続リーク、SQLインジェクション、平文パスワー
High（近日対応）	5 件	在庫キャッシュ競合、パスワード大文字無効化、削除
Medium（計画対応）	5 件	N+1クエリ、ETL全件洗い替え、在庫突合テーブル名
Low（技術負債管理）	4 件	XSLT税率ハードコード、文字化けリスク、廃止フロー
合計	**18 件**	

GitHub Issues 分析

#11: [ISSUE-211] 倉庫を切り替えると在庫数が別倉庫の値のまま表示される

- **状態**: open
- **現象**: 在庫照会画面で倉庫 A → 倉庫 B に切り替えたとき、一部の商品で倉庫 A の在庫数が表示されたまま残る。再読み込みで直ることがある。同一商品コードが複数倉庫に存在するケースで発生しやすい。
- **根本原因**:

`InventoryServlet.java:20` に `static Map<String, Integer> stockCache`
が宣言されており、キャッシュキーが **商品コード (itemCd) のみ**
で倉庫コードを含んでいない。

```java // InventoryServlet.java:20 private static Map<String, Integer> stockCache = new java.util.HashMap<String, Integer>();
```

// InventoryServlet.java:33 – 35 ← 倉庫Aで表示後にキャッシュを商品コードだけで上書き for (Map<String, Object> r : rows) { stockCache.put((String) r.get("itemCd"), (Integer) r.get("stockQty")); } ```

フロー： 1. 倉庫 A を表示 → `stockCache.put("ITM-001", 50)`（W001の在庫50） 2. 倉庫 B を表示 →
DBから正しい値を取得するが、もし例外や描画タイミングの差異でキャッシュ参照が先になると W001 の値50が返る 3. さらに `doPost`（在庫調整）では `stockCache.get(itemCd)` のみで値を取得・更新しており、キャッシュと実DB値が乖離したまま画面に反映される（`InventoryServlet.java:66 – 68`）

加えて `stockCache` はシングルトン Servlet の `static フィールド`であるにも関わらず、`HashMap`（非スレッドセーフ）を使用しているため、並行リクエスト時にデータ競合も発生しうる（`synchronized` / `ConcurrentHashMap` 未使用、検索でも確認済み）。

- **改善措置**:

1. **短期**: `stockCache` を廃止し、常に DB
から最新値を取得する。もしキャッシュが必要であればキーを `itemCd + "_" + warehouseCd` の複合キーにする。 2. **中期**: キャッシュを使うなら `ConcurrentHashMap`
に替え、スレッドセーフを保証する。 3. **抜本**: DBアクセス層を整理し、画面ごとに適切な SQL で必要な倉庫の在庫だけを取得する設計に変更する。

#12: [ISSUE-233] 受注登録を続けると「接続が取得できません」で全画面が停止する

- **状態**: open
- **現象**: 受注登録を短時間に 10〜20 回繰り返すと DB 接続エラーが発生し、在庫照会・受注一覧を含む全画面が停止する。アプリ再起動で一時復旧するが再発する。
- **根本原因**:

`DBUtil.java` の設計に3つの致命的欠陥がある。

欠陥①: 単一静的接続の使い回し（DBUtil.java:18, 33 – 37） ```java // DBUtil.java:18 private static Connection sharedConnection = null;

// DBUtil.java:33 – 37 public static Connection getConnection() throws SQLException { if (sharedConnection == null || sharedConnection.isClosed()) { sharedConnection = DriverManager.getConnection(URL, USER, PASSWORD); sharedConnection.setAutoCommit(true); } return sharedConnection; } ``` Tomcat
は多スレッドでリクエストを並行処理するが、全スレッドが同一 `Connection`
オブジェクトを共有している。複数スレッドが同一 `Connection` の `Statement`
を同時実行すると JDBC ドライバが例外を投げる。

欠陥②: `close()` が何もしない no-op（DBUtil.java:40 – 43） ```java // DBUtil.java:40 – 43 public static void close(Connection con) { // ※ 共有接続を close するとアプリ全体が止まるので、ここでは何もしない. } ``` `close()` を no-op
にした結果、呼び出し側が接続を解放したつもりでも何も起きない。

欠陥③: Statement / ResultSet のclose漏れ（OrderServlet.java:97 – 98） ```java // OrderServlet.java:97 – 98（コメントより） // st / rs は close せず GC 任せ (2008 年以来の習慣) ```` `doList` 内の `Statement` と `ResultSet`、および `fetchCustomerName` 内のリソースが GC 依存でしかクローズされない。共有接続上で Statement が溜まり続け、Oracle セッションリソースが枯渇する。

これらが組み合わさり、「受注登録を繰り返すと Connection が使用不能になり全画面停止」という事象が発生する。アプリ再起動で `sharedConnection = null` にリセットされるため一時復旧するが、根本が直っていないため再発する。

- **改善措置**:

1. **即時対応（Critical）**: `DBUtil` を廃止し、DBCP2 または HikariCP などの接続プールを導入する。`web.xml` に `<Resource>` で JNDI データソースを定義し、各クラスで `DataSource.getConnection()` を呼ぶ形に変更する。 2. **即時対応**: 全 DAO/Servlet で `try-with-resources` を使い `Connection`/`Statement`/`ResultSet` を確実にクローズする。 3. **設計見直し**: `static フィールド` での接続管理を禁止するコーディング規約を策定する。

#13: [TICKET-ASTR-024] customer_master_sync 実行後に退会済み顧客が有効として復活する

- **状態**: open
- **現象**: 日次の顧客マスタ同期フロー実行翌日、基幹側で退会・削除済みの顧客が DWH 側で「有効」として復活する。新規・更新は正しく反映されている。
- **根本原因**:

`customer_master_sync.flwx` のコンポーネント C2（Mapper）に記述されたフィルタ条件が原因。

```xml <!-- customer_master_sync.flwx:41 – 42 --> <!-- delFlg='1' は論理削除だが DWH 側に列が無いので落とす --> <filter expr="delFlg == '0'"/> ```

この `<filter>` は `delFlg == '0'`（有効レコード）のみを後続に流し、`delFlg == '1'`（退会済み）のレコードを**単に破棄している**。

後続の C3（DBOutput）は `<mode>truncate_insert</mode>` で動作する（`customer_master_sync.flwx:53`）。

```xml <!-- customer_master_sync.flwx:53 --> <mode>truncate_insert</mode> ```

フロー実行時の動作： 1. C3 がまず `DIM_CUSTOMER` テーブルを **TRUNCATE**（全件削除） 2. C2 からフィルタで `delFlg == '0'` のレコードのみ INSERT 3. 退会済み顧客（`delFlg == '1'`）は C2 で破棄されたため INSERT されない → **結果として DIM_CUSTOMER から退会済み顧客レコードが消える = 正しい動作**

ところが、ここに**別の問題**がある。OMS 側の M_CUSTOMER テーブルは `DEL_FLG CHAR(1) DEFAULT '0'` で管理されており、退会処理で `DEL_FLG='1'`

を立てるだけである。同期フローは `TRUNCATE → 有効顧客のみ INSERT`
という設計なので、理論上は翌日には退会済み顧客が消えるはずである。

しかし Issue の現象（退会済みが「有効として復活する」）が実際に発生するのは、以下のシナリオが考えられる：

- **OMS 側で退会フラグを立てる処理が別フローや手動バッチで後から `DEL_FLG='0'` にリセットされているケース**
- または **C3 の TRUNCATE が失敗・スキップされ、前回 INSERT 分（有効だった顧客）が残存するケース**（`<!-- 例外ハンドラ未定義。フロー失敗時は Warp 既定の挙動 (リトライ無し、メール無し) -->` `customer_master_sync.flwx:91`）
- さらに、DWH 側 `DIM_CUSTOMER` に `delFlg` 相当の列がなく「有効な顧客だけが格納される」仕様のため、退会後に再び `DEL_FLG='0'` が立つと次の同期で即座に復活する

コアの設計問題は**「削除情報を DWH に持ち込まないまま全件洗い替えする設計」と「例外ハンドラ未定義によりTRUNCATEだけ通過してINSERTが0件になる可能性」**の組み合わせにある。

- **改善措置**:

1. **短期**: 例外ハンドラを定義し、TRUNCATE 後に INSERT が0件だった場合はロールバック相当の処置（前回バックアップテーブルからの復元）を行う。 2. **中期**: `truncate_insert` ではなく `MERGE`（UPSERT + 論理削除反映）に切り替え、`delFlg='1'` のレコードは DWH 側でも `IS_DELETED=1` 等のフラグを立てる方式にする。 3. **根本**: DWH 側 `DIM_CUSTOMER` テーブルに `IS_DELETED` 列を追加し、基幹の `DEL_FLG` を同期させる設計に変更する。

コードレベル問題（未起票 Issue）

[Critical] 全画面にわたる SQLインジェクション脆弱性（複数箇所）

- **現象**: ユーザ入力値を直接 SQL 文字列に結合しており、悪意ある入力で任意の SQL が実行可能。
- **根本原因**:

ファイル	行	脆弱な箇所
`InventoryDAO.java`	26 – 29	`getStock()` で `itemCd` / `warehouseCd` を文字列結合
`LoginServlet.java`	33 – 40	`userId` を文字列結合（パスワードも同様）
`OrderServlet.java`	73	`customerNm` を `LIKE '%...%'` に直接結合
`OrderServlet.java`	76, 79, 82	`dateFrom` / `dateTo` / `statusCd` を直接結合
`OrderServlet.java`	113	`fetchCustomerName` で `customerCd` を直接結合
`OrderServlet.java`	125	`doDetail` で `orderNo` を直接結合
`OrderServlet.java`	146	`doCancel` で `orderNo` を直接結合

例（LoginServlet.java:36 – 40）: ```java String sql = "SELECT USER_NM, DEPT_CD FROM M_USER " + "WHERE USER_ID = '" + userId + "' " // ← 未サニタイズ + "AND PASSWORD = '" + hashed + "' " + "AND DEL_FLG = '0'"; ``` `userId` に `' OR '1'='1` を入力すると認証バイ

パスが成立する。ISSUE-156（「'」でエラー画面）もこの問題から発生している。

- **改善措置**: 全箇所で `PreparedStatement` のプレースホルダ（`?`）を使用する。動的検索条件を組み立てる `doList` も含め、文字列結合による SQL 生成を禁止する。

[Critical] DBパスワードの平文ハードコード（OMS・Asteria 両方）

- **現象**: DBパスワードがソースコードおよびXML設定ファイルに平文で埋め込まれており、リポジトリへのアクセス権があれば誰でも本番DB認証情報を取得できる。
- **根本原因**:

ファイル	行	内容
`DBUtil.java`	16	`PASSWORD = "oms_app_2014!"`
`web.xml`	24	`<param-value>oms_app_2014!</param-value>`
`jdbc_connections.xml`	12	`<password>EtlReader@2017</password>`
`jdbc_connections.xml`	21	`<password>etl_user_2018</password>`
`jdbc_connections.xml`	29	`<password>Dwh#Loader2019</password>`
`jdbc_connections.xml`	35	`<password>SvcEtl_2017!</password>`

セキュリティ部から TICKET-ASTR-031 として指摘済みだが未対応。

- **改善措置**: 環境変数または Vault（HashiCorp Vault / クラウド Secret Manager 等）に移行する。ソースコードへの平文埋め込みを禁止するシークレットスキャン（例: git-secrets, truffleHog）をCI/CDに組み込む。

[High] 在庫引き当て時の楽観排他制御欠如（ISSUE-178 継続）

- **現象**: 同一商品・倉庫に対して複数リクエストが同時に在庫引き当てを行うと、在庫数が二重に減算される（ネガティブ在庫になる可能性もある）。
- **根本原因**:

`InventoryDAO.java:82 – 103` の `decreaseStock()` メソッドは「読み取り→計算→書き込み」の3ステップで構成されているが、**排他制御が一切ない**。

```java
// InventoryDAO.java:82 – 103
int current = getStock(itemCd, warehouseCd); // ← スレッドAが50を読む
// ← スレッドBも50を読む（同時）
if (current < qty) { throw ... }
int after = current - qty; // A: 50-30=20, B: 50-20=30
// UPDATE実行 → A: STOCK=20, B: STOCK=30（最後が勝ち → 30件分だけ減った扱い）
```

テーブルの DDL コメントにも `-- ※ 楽観排他用の VERSION 列が無い（ISSUE-178 の原因と疑われる）`（schema.sql:72）と記載されている通り、`T_INVENTORY` に `VERSION` 列が存在しない。また共有接続（DBUtil.java:18）のため、`SELECT` と `UPDATE` が異なるトランザクションになるケースもある。

- **改善措置**:

1. `T_INVENTORY` に `VERSION NUMBER(10) DEFAULT 0` 列を追加し、楽観排他制御（`WHERE VERSION = ?` かつ `SET VERSION = VERSION + 1`）を実装する。
2. または `SELECT ... FOR UPDATE` による悲観排他ロックを採用する。
3.

`decreaseStock` の `UPDATE` 文を `UPDATE T_INVENTORY SET STOCK_QTY = STOCK_QTY - ? WHERE ITEM_CD=? AND WAREHOUSE_CD=? AND STOCK_QTY >= ?` の形式（アトミック更新）に変更するだけでも競合リスクを大幅に低減できる。

[High] パスワードの `toLowerCase()` による大文字認証不能（ISSUE-203）

- **現象**: ユーザがパスワードに英大文字を使用した場合、ログインできない（パスワード変更画面で登録できても認証が通らない）。
- **根本原因**: `LoginServlet.java:65` でパスワードを MD5 ハッシュする前に `toLowerCase()` を適用している。

```java // LoginServlet.java:65 byte[] b = md.digest(s.toLowerCase().getBytes("Windows-31J")); ```

パスワード `Password123` を入力すると `password123` の MD5 が計算される。パスワード変更画面でそのまま `Password123` の MD5（大文字込み）で保存していると、認証時に照合が一致しない。コード自体にも `// ※ パスワード変更画面で大文字が通らない報告 (ISSUE-203) → ここで lower してるのが原因かも` とコメントされている（LoginServlet.java:64）。

- **改善措置**:

1. **即時対応**: `toLowerCase()` を削除する。 2. **抜本対応**: MD5は暗号学的に破られており、認証には使用すべきでない。`BCrypt` または `PBKDF2WithHmacSHA256` + ソルトに切り替え、`M_USER.PASSWORD` 列も `VARCHAR2(60)` 以上に拡張する。設計書には「SHA-256 + Salt」と記載があるが実装は「MD5 ノーソルト」であり、乖離が存在する。

[High] ログイン障害時のスタックトレース漏洩

- **現象**: ログイン処理で例外が発生した場合、スタックトレース全文が HTTP レスポンスとしてブラウザに返送される。
- **根本原因**: `LoginServlet.java:55 – 57`

```java // LoginServlet.java:55 – 57 res.getWriter().println("Login error: " + e.getMessage()); for (StackTraceElement el : e.getStackTrace()) { res.getWriter().println(" at " + el); } ```

内部クラス名・パッケージ名・ファイルパスがブラウザに露出し、攻撃者の情報収集に利用される。

- **改善措置**: スタックトレースをブラウザへ出力する処理を削除し、サーバ側ログ（`log4j` 等）にのみ記録する。ユーザには汎用エラーページ（`/error.jsp`）を表示する。

[High] XSS（クロスサイトスクリプティング）脆弱性 — 在庫画面（ISSUE-217 対応）

- **現象**: `inventory.jsp` の取引先プルダウンで、取引先名に `<script>` が含まれるデータがそのまま HTML 出力されてスクリプトが実行される。
- **根本原因**: `inventory.jsp:44`

```jsp // inventory.jsp:44 // ※ CUSTOMER_NM には自由入力欄に <script> を貼られたデータが残存しているケースあり (ISSUE-217) out.println("<option value= \ "" + e.getKey() + " \ ">" + e.getValue() + "</option>"); ```

`e.getValue()`（取引先名）が HTML エスケープされずに出力される。また `order_list.jsp` の取引先名列（行63: `<td><%= customerNm %></td>`）も同様に未エスケープ。

- **改善措置**: JSP 内の全出力箇所で `JSTL <c:out value="${...}"/>` または `org.apache.commons.lang.StringEscapeUtils.escapeHtml()` を使用する。スクリプトレット (`<% %>`) を排除し、EL式 + JSTL に移行する。

[Medium] 受注一覧の N+1 クエリ問題（ISSUE-142 対応）

- **現象**: 受注一覧表示が10,000件超で30秒以上かかる。
- **根本原因**: `OrderServlet.java:96 – 99`

```java // OrderServlet.java:96 – 99 // 取引先名を 1 行ずつ取得して付与（テンプレート側で楽するため） row.put("customerNm", fetchCustomerName(rs.getString("CUSTOMER_CD"))); ```

`fetchCustomerName`（OrderServlet.java:108 – 115）は受注1件ごとに `M_CUSTOMER` テーブルへ SQL を1回発行する。1万件の受注があれば1万+1回の SQLが実行される（N+1問題）。さらに `M_CUSTOMER` への SQL 内でも `customerCd` が文字列結合で SQLインジェクション可能（OrderServlet.java:113）。

加えて `T_ORDER` に `ORDER_DT` / `CUSTOMER_CD` のインデックスが無い（schema.sql:51 コメント）。

- **改善措置**:

1. `T_ORDER` と `M_CUSTOMER` を `JOIN` した一本の SQL に変更し、N+1を解消する。 2. `T_ORDER.ORDER_DT` と `T_ORDER.CUSTOMER_CD` にインデックスを追加する。

[Medium] order_etl_daily の LookupDB キャッシュ無効設定による大量 SQL 発行（TICKET-ASTR-012 関連）

- **現象**: `order_etl_daily` が件数増により2時間超かかり、後続フローと競合する。
- **根本原因**: `order_etl_daily.flwx:52`

```xml <!-- order_etl_daily.flwx:52 --> <cache enabled="false"/> ```

LookupDB コンポーネント O2 でキャッシュが無効化されており、受注1件ごとに `M_CUSTOMER` テーブルへ SQL を1回発行する。現在は数十万件の受注データに対して実行されており、LookupDB の SQL だけで総時間の大半を占めていると推定される（フロー内コメントにも明記）。
```

また、受注データは `WHERE` 条件なし全件取得（order_etl_daily.flwx:26 – 34）であり、データが増え続けるにつれてさらに悪化する。

- **改善措置**:

1. **即時**: `<cache enabled="true"/>` に変更し、`M_CUSTOMER` の参照結果をメモリキャッシュする（取引先マスタは日次同期前に確定しているため安全）。 2. **中期**: 全件洗い替えから差分更新（`UPD_DT` 基準の増分抽出）に切り替える。 3. `cron="0 30 2 * * ?"` (02:30) で FLW-002 が開始し2時間超かかると `cron="0 0 4 1 * ?"` (04:00) の FLW-003 と重複するため、スケジュール見直しも必要。

[Medium] order_etl_daily の TRUNCATE→BULK INSERT 無ロールバック設計

- **現象**: BULK INSERT が失敗すると `FACT_ORDER` テーブルが空のまま朝になる（2022-08 と 2023-12 に障害実績あり）。
- **根本原因**: `order_etl_daily.flwx:81 – 91`

```xml <sql>TRUNCATE TABLE FACT_ORDER</sql> <!-- ← 即時コミット、ロールバック不可 --> <sql>BULK INSERT FACT_ORDER ...</sql> <!-- ← 失敗すると FACT_ORDER が空に --> ```

TRUNCATE は DDL 操作のため自動コミットされ、後続の BULK INSERT が失敗してもロールバックできない。フロー内にも `<!-- ロールバック設計は無し -->` と明記されている。

- **改善措置**: TRUNCATE→INSERT のパターンを廃止し、ステージングテーブル（`FACT_ORDER_STG`）へ先に INSERT し、成功後に `FACT_ORDER` をスワップ（`RENAME`）する Blue/Green 方式か、MERGE 文による差分更新に変更する。

[Medium] inventory_reconcile の W003 倉庫テーブル名誤り

- **現象**: W003 倉庫の在庫突合で常に差分が全件扱いになる（W003 の差分レポートが信用できない）。
- **根本原因**: `inventory_reconcile.flwx:57 – 59`

```xml <!-- inventory_reconcile.flwx:57 – 59 --> <!-- ※ 2022 年に追加された W003 のみ DWH 側テーブル名が違う (歴史的経緯) だが、ここでは DIM_INVENTORY を見ているので常に 0 件返る → 差分が全件扱いになる --> <query>SELECT ITEM_CODE, STOCK_QTY FROM DIM_INVENTORY WHERE WAREHOUSE='W003'</query> ```

W003 の DWH 側テーブルは別名（`DIM_INVENTORY_W003`）だが、フローは他倉庫と同じ `DIM_INVENTORY` を参照している。W003 の DWH 側データは常に0件返るため、突合結果が常に「全件差分あり」となる。

- **改善措置**: W003_DWH コンポーネントのクエリを `DIM_INVENTORY_W003` の正しいテーブル/ビュー名に修正する。長期的には倉庫ごとのテーブル分割運用を統廃合し、`WAREHOUSE` 列で区別する単一テーブル設計に標準化する。

[Medium] customer_master_sync の SMTP 件名が常に `[FAILED]`（TICKET-ASTR-018）

- **現象**: 同期フローが正常終了しても送信メールの件名が `[FAILED]` になる。監視担当者が誤報アラートとして無視するようになり、実際の障害を見落とすリスクがある。
- **根本原因**: `customer_master_sync.flwx:78`

```xml
<!-- customer_master_sync.flwx:78 -->
<subject>[FAILED] customer_master_sync done runId=${vars.runId} count=${vars.recCount}</subject>
```

件名が `[FAILED]` にハードコードされており、動的に成功/失敗を切り替えるロジックが実装されていない（実装途中で放棄されたとコメントに明記）。

- **改善措置**: Asteria Warp の条件分岐コンポーネントを使い、正常終了パスでは `[OK]`、例外ハンドラ（未定義のため同時に実装が必要）では `[FAILED]` の件名になるよう分岐させる。

[Low] XSLT 内の税率ハードコードと remarks 依存判定

- **現象**: 税率判定が `remarks` フィールドのキーワード（「軽減」「非課税」）依存で、事業部が別の文言を入力すると誤った税率が適用される。税率改定のたびにXSLTを直書きしており、多重 `if-else` が層状に残存。
- **根本原因**: `order_transform.xsl:26 – 34`

```xml
<xsl:when test="contains(remarks,'軽減')">0.08</xsl:when>
<xsl:when test="contains(remarks,'非課税')">0.00</xsl:when>
<xsl:otherwise>0.10</xsl:otherwise>
```

また TOTAL_AMT が店舗によって税込/税抜き混在（`order_transform.xsl:44 – 48` コメント参照）のため、税抜金額の逆算も誤差を生む可能性がある。

- **改善措置**: OMS 側に `TAX_TYPE` 列（標準/軽減/非課税）を追加し、XSLT での推測判定を廃止する。税率はコード値から引くテーブル参照方式にする。

[Low] セッションタイムアウト 240分（4時間）設定

- **現象**: 設計書には「30分」と記載されているが実装は240分。長時間離席時にセッションが継続し、なりすまし攻撃のリスクが高まる。
- **根本原因**: `web.xml:55` `<session-timeout>240</session-timeout>`
- **改善措置**: 業務要件に合わせて30〜60分程度に短縮する。アイドルタイムアウトと絶対タイムアウトを分離して設定する。

[Low] FLW-099 廃止済みフローの残存

- **現象**: `project.xml` に `FLW-099（legacy_csv_export.flwx）` が `enabled="false"` で残存しており、参照ファイルが存在しないにもかかわらず定義だけが残っている（管理上の混乱要因）。
- **根本原因**: `project.xml:40 – 43`

- **改善措置**: `FLW-099` のエントリを削除し、廃止済みフローの定義を整理する。

設計書と実装の乖離

#	設計書	設計書の記載	実装の実態	該当ファイル:行
1	OMS 基本設計書.docx	DB接続プール（DBCP）	`使用 ConnectionManager` で単一静的接続を使い回し	`DbConnectionManager.getConnection()`
2	OMS 基本設計書.docx	セッションタイムアウト 30分	web.xml に `<session-timeout>30</session-timeout>`	`web.xml:34`
3	OMS 基本設計書.docx	パスワードハッシュ SHA256 + Salt	`MessageDigest.getInstance("MD5")`	`LoginServlet.java:62,65`
4	OMS テーブル定義書.xlsx	`T_INVENTORY.VERSION` 列あり	DDL に VERSION 列なし	`schema.sql:65 – 71`
5	OMS 機能・画面一覧.xlsx	アカウントロック機能	実装済	ソースに存在しない LoginServlet 全体
6	OMS 機能・画面一覧.xlsx	文字コード処理 実装済	`InventoryServlet.java:61`	`InventoryServlet.java:61 setCharacterEncoding("UTF-8")`
7	OMS IF仕様書.xlsx	廃止済 EDI 連携「稼働中」	実コードに EDI 連携実装なし（廃止済）	
8	Asteria 基本設計書.docx	差分更新（増分）連携	全件 TRUNCATE → INSERT	`FLW-099_sync.flwx:53`, `FLW-099_sync.flwx:61`
9	Asteria 基本設計書.docx	リトライ機能あり	例外ハンドラ未定義、リトライなし	`FLW-099_master_sync.flwx:91`
10	Asteria 基本設計書.docx	パスワード暗号化保管	`jdbc_connections.xml` に平文パスワード	`jdbc_connections.xml:12,21,29,37`
11	Asteria DWH連携テーブル定義書.docx	`INVENTORY_W003` を別テーブルで運用	`FLW-099_sync.flwx:59`	`FLW-099_sync.flwx:59`
12	Asteria フロー一覧.xlsx	`FLW-099`「現役」	`project.xml` で `enabled="false"`	`project.xml:40` 参照ファイル不在
13	Asteria 連携IF一覧.xlsx	税率値域が古い記載	XSLT に軽減税率 0.08 / 標準 0.10 の 2 値	`tax_rate.xslt:3` （古い税率体系）

優先度マトリクス

#	問題	影響度	緊急度	推奨対応期限
1	DB接続プール欠如	高（接続リーク → 全画面停止）[ISSUE-233]	高	**即時（今週中）**
2	SQLインジェクション	高（認証バイパス含む）	高（全画面）	**即時（今週中）**
3	DBパスワード平文	高（ハードコード（OMS/Asteria）	高	**即時（今週中）**
4	在庫二重減算（楽観排他欠如）[ISSUE-17	高	継続]	**即時（今週中）**
5	在庫キャッシュの倉庫混在バグ [ISSUE-241]	中		2週間以内
6	パスワード大文字認証不能 [ISSUE-203	中	継続]	2週間以内
7	退会済み顧客の DWH 復活 [TICKET-ASTR-024]	高	中	2週間以内
8	XSS（在庫・受注一覧画面）[ISSUE-217	高	継続]	2週間以内
9	スタックトレースのブラウザ漏洩	中	中	2週間以内
10	受注一覧 N+1 クエリ [ISSUE-142継続]	中	中	1ヶ月以内
11	order_etl_daily LookupDB キャッシュ無効 [TICKET-ASTR-012]	高	中	1ヶ月以内
12	TRUNCATE→BULK INSERT ロールバック不可	高	低	1ヶ月以内
13	inventory_reconcile W003 テーブル名誤り	中	中	1ヶ月以内
14	SMTP件名常時[FAILED] [TICKET-ASTR-018]	低	中	1ヶ月以内

#	問題	影響度	緊急度	推奨対応期限
15	XSLT 税率ハードコード	中	低	四半期以内
16	セッションタイムアウト 240分（設計書と乖離）	低	低	四半期以内
17	廃止フロー FLW-099 残存	低	低	四半期以内
18	MD5→BCrypt/PBKDF2 移行（設計書と乖離、移行コスト高）	高	中	ロードマップ策定後

付録: 調査対象ファイル一覧

ファイル	種別	主な問題
`DBUtil.java`	Java	静的接続使い回し、no-op close、平文パスワード
`LoginServlet.java`	Java	SQLi、MD5ノーソルト、toLowerCase、スタックト
`OrderServlet.java`	Java	SQLi多数、N+1、Statement/RS未クローズ
`InventoryServlet.java`	Java	倉庫混在キャッシュ、非スレッドセーフ HashMap
`InventoryDAO.java`	Java	SQLi、楽観排他欠如、fetchItemName リソース漏れ
`inventory.jsp`	JSP	XSS、JSP内DB直アクセス
`order_list.jsp`	JSP	未エスケープ出力
`schema.sql`	SQL	VERSION列欠如、インデックス欠如
`web.xml`	XML	平文パスワード、セッション240分
`DBUtil.java`	Java	平文パスワードDBUtil.java:16
`jdbc_connections.xml`	XML	全接続先パスワード平文（TICKET-ASTR-031）
`customer_master_sync.flwx`	FLWX	退会顧客復活、SMTP件名固定、例外ハンドラ未定義
`order_etl_daily.flwx`	FLWX	LookupDB キャッシュ無効、TRUNCATE無ロールバ
`inventory_reconcile.flwx`	FLWX	W003テーブル名誤り、直列処理
`order_transform.xsl`	XSL	税率ハードコード、remarks依存判定、税込/税抜混在
`project.xml`	XML	FLW-099廃止済み残存